Name: Oisin Godden

Student number: 24271187

The Docklands Wildlife Tracker is an application that allows community members in the Docklands to log, view, and verify wildlife sightings. The application identifies the lack of local species data by providing a digital platform where citizens can contribute observations which are then stored and organized using data structures.

Core Functionality

The application is built around a WildlifeSighting class, which serves as the main data model. Each sighting object contains the following attributes:

- species (String): The type of species observed
- location (String): Where the sighting occurred
- observer (String): The name of the person reporting it
- timestamp (Date): When the sighting happened
- verified (Boolean): Whether the sighting has been confirmed

Data Storage and Management
All sightings are stored in a **Singly Linked List** called SightingList. This list implements:

- addSighting(): Creates a new node containing the sighting object and appends it to the end of the list
- removeSighting(): Locates a sighting by index or search criteria and adjusts the node pointers to bypass the target node
- getSighting(): Traverses the list from the head node until the requested index is reached
- getAllSightings(): Iterates through the list and collects all sighting objects for display

Verification Workflow
When a user submits a new sighting:

1. The sighting object is added to the main SightingList with verified = false
2. The index position of that sighting is added to a Queue data structure

Queueing

The queue operates on a FIFO (First-In-First-Out) basis. When a verified user or expert accesses the verification function, the application:

- Calls poll() on the queue to retrieve the next index

- Retrieves the corresponding sighting from the linked list using get(index)
- Calls the verify() method on that sighting object (implemented from the Verifiableinterface)
- Updates the display to reflect the change

Stacking

The stack operates on a LIFO (Last-In-First-Out). The application includes an "Undo" function that calls pop() on the stack to retrieve the most recent action and reverse it. This provides users with a safety net when mistakes are made.

Interfaces

Two interfaces ensure consistent behaviour across the application:

- Verifiable: Requires verify() method that changes a sighting's status from pending to verified
- Displayable: Requires getSummary() method that returns a formatted string representation of a sighting

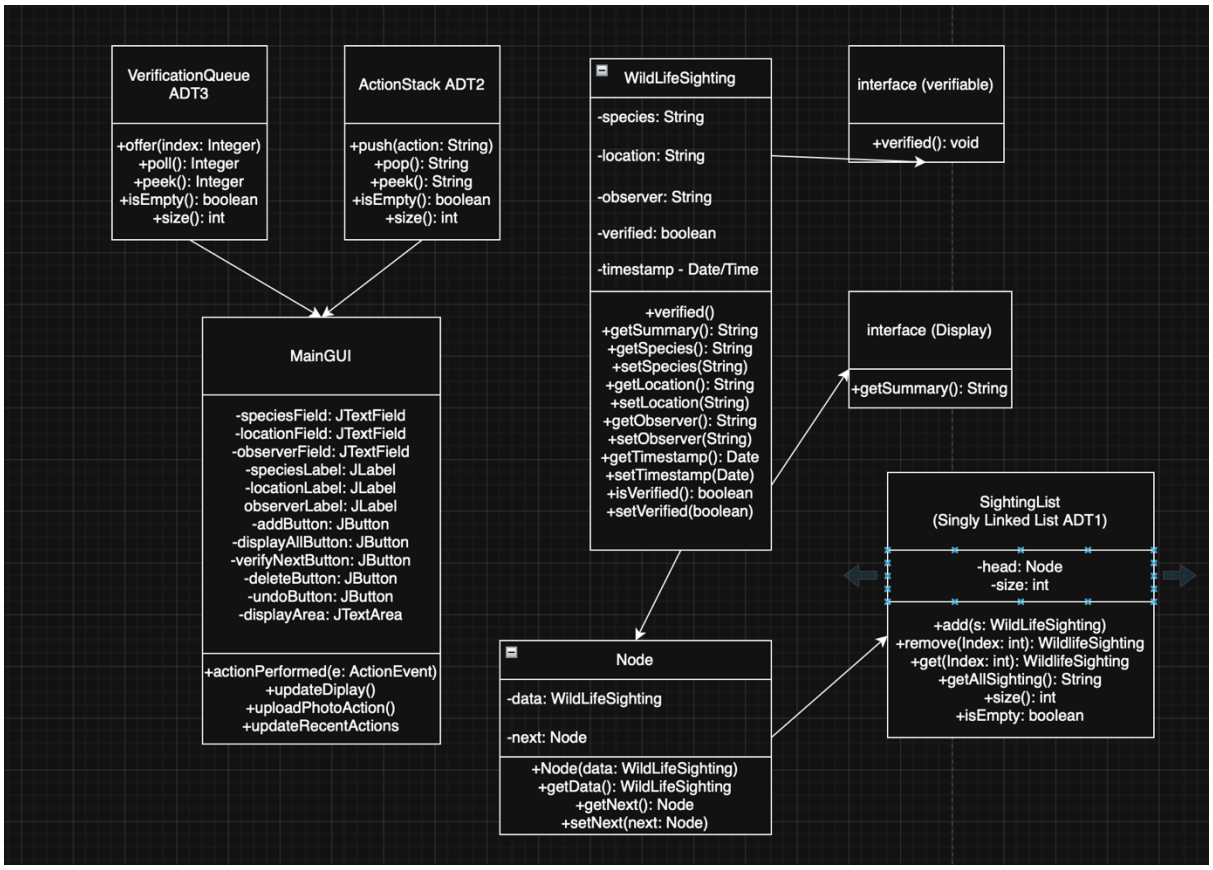
GUI

The GUI provides:

- Input fields for species, location, and observer name
- An "Add Sighting" button that creates a new WildlifeSighting object and adds it to both the linked list and verification queue
- A "Display All" button that iterates through the linked list and calls getSummary() on each sighting
- A "Verify Next" button that processes the next item in the queue and updates its status
- A "Delete" button that removes a selected sighting from the linked list
- An “Undo” that reverses the most recent action

How This Fits the Theme

The application supports the Smart Docklands innovative objective of "testing al resilience" by providing a manual survey to track biodiversity over time. If a new green roof is installed, users can log sightings before and after uploading. Planners could later analyse this data to determine whether the intervention attracted more wildlife.



GitHub Link:

https://github.com/oisingodden/CA1-Data-Structures-and-Algorithms